УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 11

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С БИНАРНЫМИ ФАЙЛАМИ И ФАЙЛАМИ EXCEL

ЛАБОРАТОРНАЯ РАБОТА № 11.1

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С БИНАРНЫМИ ФАЙЛАМИ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для работы с бинарными файлами

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

Очень часто нужно сохранить какие-либо данные. Для этого существуют два способа: **запись в файл** и **сохранение в базу данных**. Первый способ используется при сохранении информации небольшого объема. Если объем велик, то лучше (и удобнее) воспользоваться базой данных.

4.1 Открытие файла в Python

Прежде чем работать с файлом, необходимо создать объект файла с помощью функции **open ()**. Функция имеет следующий формат:

```
open(<Путь  к файлу>[, mode = 'r'] [, buffering=-1] [, encoding=None] [,
   errors=None][, newline=None][, closefd=True])
```

В первом параметре указывается путь к файлу. Путь может быть абсолютным или относительным. При указании абсолютного пути в **Windows** следует учитывать, что в **Python** слеш является специальным символом. По этой причине слеш необходимо удваивать или вместо обычных строк использовать неформатированные строки.

Пример:

```
"C:\\temp\\new\\file.txt"    # Правильно
'C:\\temp\\new\\file.txt'
r"C:\temp\new\file.txt"      # Правильно
'C:\\temp\\new\\file.txt'
"C:\temp\new\file.txt"       # Неправильно!!!
'C:\temp\new\x0cile.txt'
```

Обратите внимание на последний пример. В этом пути из-за того, что слеши не удвоены, возникло присутствие сразу трех специальных символов:

\t, **\n** и **\f** (отображается как **\x0c**). После преобразования этих специальных символов путь будет выглядеть следующим образом: . , . .

C:<Табуляция>emp<Перевод строки>ew<Перевод формата>ile.txt

Если такую строку передать в функцию **open ()**, то это приведет к исключению **OSError**:

```
open("C:\temp\new\file.txt")
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    open("C:\temp\new\file.txt")
OSError: [Errno 22] Invalid argument: 'C:\temp\new\x0cile.txt'
```

Вместо абсолютного пути к файлу можно указать относительный путь. В этом случае путь определяется с учетом местоположения текущего рабочего каталога. Относительный путь будет автоматически преобразован в абсолютный путь с помощью функции **abspath ()** из модуля **os.path**. Возможны следующие варианты:

– если открываемый файл находится в текущем рабочем каталоге, то можно указать только название файла.

Пример:

```
import os.path  #  Подключаем модуль
# Файл в текущем рабочем каталоге (C:\Python34\)
print(os.path.abspath(r"file.txt"))
'C:\\Python34\\file.txt'
```

– если открываемый файл расположен во вложенной папке, то перед названием файла приводятся названия вложенных папок через слеш.

Примеры:

```
# Открываемый файл в C:\Python34\folder1\
print(os.path.abspath (r"folder1/file.txt"))
'C:\\Python34\\folder1\\file.txt'
# Открываемый файл в C:\Python34\folder1\folder2\
print( os.path.abspath(r"folder1/folder2/file.txt"))
'C:\\Python34\\folder1\\folder2\\file.txt'
```

– если папка с файлом расположена ниже уровнем, то перед названием файла указываются две точки и слеш (" .. /").

Пример:

```
# Открываемый файл в C:\
print(os.path.abspath (r"../file.txt"))
'C:\\file.txt'
```

– если в начале пути расположен слеш, то путь отсчитывается от корня диска. В этом случае местоположение текущего рабочего каталога не имеет значения.

Примеры:

```
# Открываемый файл в C:\book\folder1\
print(os.path.abspath (r"/book/folder1/file.txt"))
'C:\\book\\folder1\\file.txt'
# Открываемый файл в C:\book\folder1\folder2\
```

```
print(os.path.abspath (r"/book/folder1/folder2/file.txt"))
'C:\\book\\folder1\\folder2\\file.txt'
```

Как можно видеть, в абсолютном и относительном путях можно указать как прямые, так и обратные слеши. Все они будут автоматически преобразованы с учетом значения атрибута **sep** из модуля **os.path**. Значение этого атрибута зависит от используемой операционной системы. Выведем значение атрибута **sep** в операционной системе **Windows**:

```
print(os.path.sep)
'\\'
print(os.path.abspath (r"C:/book/folder1/file.txt"))
'C:\\book\\folder1\\file.txt'
```

При использовании относительного пути необходимо учитывать местоположение текущего рабочего каталога, т. к. рабочий каталог не всегда совпадает с каталогом, в котором находится исполняемый файл. Если файл запускается с помощью двойного щелчка на его значке, то каталоги будут совпадать. Если же файл запускается из командной строки, то текущим рабочим каталогом будет каталог, из которого запускается файл.

Рассмотрим все это на примере, для чего в каталоге **C:\book** создадим следующую структуру файлов:

```
C:\book\
  test.py
  folder1\
    __init__.py
    module1.py
```

Содержимое файла **C:\book\test.py**:

```
# -*- coding: utf-8 -*-
import os, sys
print("%-25s%s" % ("Файл:", os.path.abspath(__file__)))
print("%-25s%s" % ("Текущий рабочий каталог:", os.getcwd()))
print("%-25s%s" % ("Каталог для импорта:", sys.path[0]))
print("%-25s%s" % ("Путь к файлу:", os.path.abspath("file.txt")))
print("-" * 40)
import folder1.module1 as m
m.get_cwd()
```

Файл **C:\book\folder1__init__.py** создаем пустым. Этот файл указывает интерпретатору **Python**, что данный каталог является пакетом с модулями.

Содержимое файла **C:\book\folder1\module1.py**:

```
# -*- coding: utf-8 -*-
import os, sys
def get_cwd():
    print("%-25s%s" % ("Файл:", os.path.abspath(__file__)))
    print("%-25s%s" % ("Текущий рабочий каталог:", os.getcwd()))
    print("%-25s%s" % ("Каталог для импорта:", sys.path[0]))
    print("%-25s%s" % ("Путь к файлу:", os.path.abspath("file.txt")))
```

Запускаем командную строку, переходим в каталог C:\book и запускаем файл test.py, результат представлен на рисунке 1:

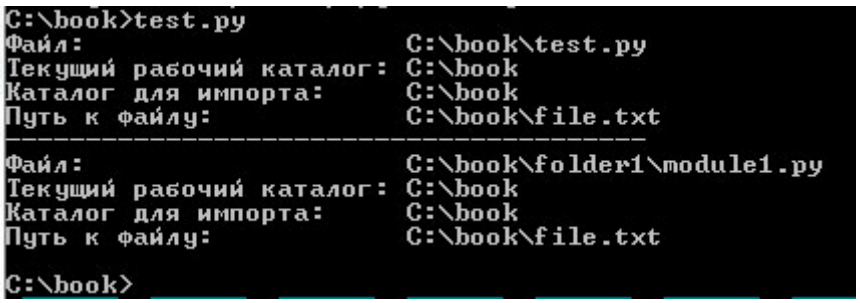


Рисунок 1 – Результат выполнения файла test.py

В этом примере текущий рабочий каталог совпадает с каталогом, в котором расположен файл **test.py**. Однако обратите внимание на текущий рабочий каталог внутри модуля **module1.py**. Если внутри этого модуля в функции **open()** указать название файла без пути, то поиск файла будет произведен в каталоге **C:\book**, а не **C:\book\folder1**.

Теперь перейдем в корень диска C: и опять запустим файл test.py, результат представлен на рисунке 2:

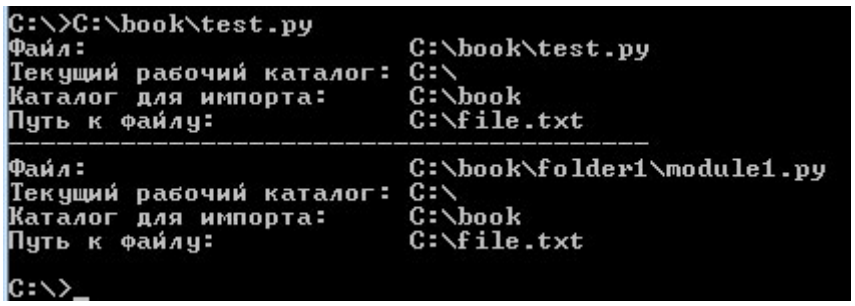


Рисунок 2 – Результат выполнения файла test.py

В этом случае текущий рабочий каталог не совпадает с каталогом, в котором расположен файл **test.py**. Если внутри файлов **test.py** и **module1.py** в функции **open()** указать название файла без пути, то поиск файла будет производиться в корне диска **C:**, а не в каталогах с этими файлами.

Чтобы поиск файла всегда производился в каталоге с исполняемым файлом, необходимо этот каталог сделать текущим с помощью функции **chdir()** из модуля **os**. Для примера изменим содержимое файла **test.py**:

```
# -*- coding: utf-8 -*-
import os, sys
# Делаем каталог с исполняемым файлом текущим
os.chdir (os.path.dirname(os.path.abspath(__file__)))
print("%-25s%s" % ("Файл:", __file__))
print("%-25s%s" % ("Текущий рабочий каталог:", os.getcwd()))
print("%-25s%s" % ("Каталог для импорта:", sys.path[0]))
```

```
print("%-25s%s" % ("Путь к файлу:", os.path.abspath("file.txt")))
```

Обратите внимание на четвертую строку. С помощью атрибута **file_**получаем путь к исполняемому файлу вместе с названием файла. Атрибут **file** не всегда содержит полный путь к файлу. Например, если запуск осуществляется следующим образом:

C:\book>C:\Python34\python test.py,

то атрибут будет содержать только название файла без пути. Чтобы всегда получать полный путь к файлу, следует передать значение атрибута в функцию abspath() из модуля os.path. Далее извлекаем путь (без названия файла) с помощью функции dirname() и передаем его функции chdir(). Теперь, если в функции open() указать название файла без пути, то поиск будет производиться в каталоге с этим файлом. Запустим файл test.py с помощью командной строки, результат представлен на рисунке 3:

```
C:\>C:\book\test.py
Файл:                     C:\book\test.py
Текущий рабочий каталог:  C:\book
Каталог для импорта:      C:\book
Путь к файлу:             C:\book\file.txt

C:\>
```

Рисунок 3 – Результат выполнения файла test.py

Функции, предназначенные для работы с каталогами, рассмотрим подробно в следующих шагах. Сейчас же важно запомнить, что текущим рабочим каталогом будет каталог, из которого запускается файл, а не каталог, в котором расположен исполняемый файл. Кроме того, пути поиска файлов не имеют никакого отношения к путям поиска модулей.

Необязательный параметр **mode** в функции **open()** может принимать следующие значения:

- **r - только чтение (значение по умолчанию)**. После открытия файла указатель устанавливается на начало файла. Если файл не существует, возбуждается исключение **FileNotFoundError**;
- **r+ - чтение и запись**. После открытия файла указатель устанавливается на начало файла. Если файл не существует, то возбуждается исключение **FileNotFoundError**;
- **w - запись**. Если файл не существует, он будет создан. Если файл существует, он будет перезаписан. После открытия файла указатель устанавливается на начало файла;
- **w+ - чтение и запись**. Если файл не существует, он будет создан. Если файл существует, он будет перезаписан. После открытия файла указатель устанавливается на начало файла;
- **a - запись**. Если файл не существует, он будет создан. Запись осуществляется в конец файла. Содержимое файла не удаляется;
- **a+ - чтение и запись**. Если файл не существует, он будет создан. Запись осуществляется в конец файла. Содержимое файла не удаляется;

– **x - создание файла для записи**. Если файл уже существует, возбуждается исключение **FileExistsError**;

– **x+ -** создание файла для чтения и записи. Если файл уже существует, возбуждается исключение **FileExistsError**.

После указания режима может следовать модификатор: '

– **b - файл будет открыт в бинарном режиме**. Файловые методы принимают и возвращают объекты типа **bytes**;

– **t - файл будет открыт в текстовом режиме** (значение по умолчанию в **Windows**). Файловые методы принимают и возвращают объекты типа **str**. В этом режиме будет автоматически выполняться обработка символа конца строки - так, в **Windows** при чтении вместо символов **\r\n** будет подставлен символ **\n**.

Для примера создадим файл **file.txt** и запишем в него две строки:

```
f = open (r"file.txt", "w") # Открываем файл на запись
print(f.write ("String1\nString2")) # Записываем две строки в файл
15
f.close()                   # Закрываем файл
```

Поскольку нужно указать режим **w**, то если файл не существует, он будет создан, а если существует, то будет перезаписан.

Теперь выведем содержимое файла в бинарном и текстовом режимах:

```
# Бинарный режим (символ \r остается)
with open (r"file.txt", "rb") as f:
for line in f:
        print(repr(line))
b'String1\r\n'
b'String2'
# Текстовый режим (символ \r удаляется)
with open (r"file.txt", "r") as f:
 for line in f:
        print(repr(line))
'String1\n'
'String2'
```

Для ускорения работы производится буферизация записываемых данных. Информация из буфера записывается в файл полностью только в момент закрытия файла или после вызова функции или метода **flush()**. В необязательном параметре **buffering** можно указать размер буфера. Если в качестве значения указан 0, то данные будут сразу записываться в файл (значение допустимо только в бинарном режиме). Значение 1 используется при построчной записи в файл (значение допустимо только в текстовом режиме), другое положительное число задает примерный размер буфера, а отрицательное значение (или отсутствие значения) означает установку размера, применяемого в системе по умолчанию. По умолчанию текстовые файлы буферизуются построчно, а бинарные - частями, размер которых интерпретатор выбирает самостоятельно в диапазоне от 4096 до 8192 байтов.

При использовании текстового режима (задается по умолчанию) при чтении производится попытка преобразовать данные в кодировку **Unicode**, а при

записи выполняется обратная операция - строка преобразуется в последовательность байтов в определенной кодировке. По умолчанию назначается кодировка, применяемая в системе. Если преобразование невозможно, то возбуждается исключение. Указать кодировку, которая будет использоваться при записи и чтении файла, позволяет параметр **encoding**. Для примера запишем данные в кодировке **UTF-8**:

```
f = open(r"file.txt","w", encoding="utf-8")
print(f.write ("Строка"))  # Записываем строку в файл
6
f.close()          # Закрываем файл
```

Для чтения этого файла следует явно указать кодировку при открытии файла:

```
with open (r"file.txt", "r", encoding="utf-8") as f:
 for line in f:
          print(line)
Строка
```

При работе с файлами в кодировках **UTF-8**, **UTF-16** и **UTF-32** следует учитывать, что в начале файла могут присутствовать служебные символы, называемые сокращенно **BOM** (**Byte Order Mark - метка порядка байтов**). Для кодировки **UTF-8** эти символы являются необязательными, и в предыдущем примере они не были добавлены в файл при записи. Чтобы символы были добавлены, в параметре **encoding** следует указать значение **utf-8-sig**. Запишем строку в файл в кодировке **UTF-8** с **BOM**:

```
f = open(r"file.txt","w", encoding="utf-8-sig")
print(f.write ("Строка"))  # Записываем строку в файл
6
f.close()          # Закрываем файл
```

Теперь прочитаем файл с разными значениями в параметре **encoding:**

```
with open (r"file.txt", "r", encoding="utf-8") as f:
 for line in f:
     print(repr(line))
'\ufeffСтрока'
with open (r"file.txt", "r", encoding="utf-8-sig") as f:
 for line in f:
     print(repr(line))
Строка
```

В первом примере нужно указать значение **utf-8**, поэтому маркер **BOM** был прочитан из файла вместе с данными. Во втором примере указано значение **utf-8-sig**, поэтому маркер **BOM** не попал в результат. Если неизвестно, есть ли маркер в файле, и необходимо получить данные без маркера, то следует всегда указывать значение **utf-8-sig** при чтении файла в кодировке **UTF-8**.

Для кодировок **UTF-16** и **UTF-32** маркер **BOM** является обязательным. При указании значений **utf-16** и **utf-32** в параметре **encoding** обработка маркера производится автоматически. При записи данных маркер автоматически

вставляется в начало файла, а при чтении он не попадает в результат. Запишем строку в файл, а затем прочитаем ее из файла:

```
with open (r"file.txt", "w", encoding="utf-16") as f:
print(f.write("Строка"))
6
with open (r"file.txt", "r", encoding="utf-16") as f:
for line in f:
    print(repr(line))
Строка
```

При использовании значений **utf-16-le**, **utf-16-be**, **utf-32-le** и **utf-32-be** маркер **BOM** необходимо самим добавить в начало файла, а при чтении удалить его.

В параметре **errors** можно указать уровень обработки ошибок. Возможные значения:

- **"strict"** - при ошибке возбуждается исключение **ValueError** (значение по умолчанию),
- **"replace"** - неизвестный символ заменяется символом вопроса или символом с кодом **\ufffd**,
- **"ignore"** - неизвестные символы игнорируются,
- **"xmlcharrefreplace"** - неизвестный символ заменяется последовательностью **&#xxxx;** и
- **"backslashreplace"** - неизвестный символ заменяется последовательностью **\uxxxx**.

Параметр **newline** задает режим обработки символов конца строк. Поддерживаемые им значения таковы:

– **None** (значение по умолчанию) - выполняется стандартная обработка символов конца строки. Например, в **Windows** при чтении символы **\r\n** преобразуются в символ **\n**, а при записи производится обратное преобразование;

– **"" (пустая строка)** - обработка символов конца строки не выполняется;

– **"<Специальный символ>"** - указанный специальный символ используется для обозначения конца строки, и никакая дополнительная обработка не выполняется. В качестве специального символа можно указать лишь **\r\n**, **\r** и **\n**.

4.2 Методы для работы с файлами в Python

После открытия файла функция **open ()** возвращает объект, с помощью которого производится дальнейшая работа с файлом. Тип объекта зависит от режима открытия файла и буферизации. Рассмотрим основные методы:

- **close ()** - закрывает файл. Так как интерпретатор автоматически удаляет объект, когда на него отсутствуют ссылки, в небольших программах можно явно не закрывать файл. Тем не менее, явное закрытие файла является признаком хорошего стиля программирования. Кроме того, при наличии незакрытого файла генерируется предупреждающее сообщение: **"ResourceWarning: unclosed file"**.

Язык **Python** поддерживает **протокол менеджеров контекста**. Этот протокол гарантирует закрытие файла вне зависимости от того, произошло исключение внутри блока кода или нет.

Пример:

```
with open(r"file.txt", "w", encoding="cp1251") as f:
    f.write("Строка") # Записываем строку в файл
# Здесь файл уже закрыт автоматически
```

– **write (<Данные>)** - записывает строку или последовательность байтов в файл. Если в качестве параметра указана строка, то файл должен быть открыт в текстовом режиме. Для записи последовательности байтов необходимо открыть файл в бинарном режиме. Помните, что нельзя записывать строку в бинарном режиме и последовательность байтов в текстовом режиме. Метод возвращает количество записанных символов или байтов. Пример записи в файл:

```
# Текстовый режим
f = open (r"file.txt", "w", encoding="cp1251")
print(f.write("Строка1\nСтрока2")) # Записываем строку в файл
15
f.close ()  # Закрываем файл
# Бинарный режим
f = open (r"file.txt", "wb")
print( f.write (bytes ("Строка 1\nСтрока2", "cp1251") ))
15
f.write (bytearray ("\nСтрока3", "cp1251"))
8
f.close ()
```

– **writelines(<Последовательность>)** - записывает последовательность в файл. Если все элементы последовательности являются строками, то файл должен быть открыт в текстовом режиме. Если все элементы являются последовательностями байтов, то файл должен быть открыт в бинарном режиме. Пример записи элементов списка:

```
# Текстовый режим
f = open(r"file.txt", "w", encoding="cp1251")
f.writelines( ["Строка1\n", "Строка2"])
f.close ()
# Бинарный режим
f = open(r"file.txt", "wb")
arr = [bytes ("Строка1\n", "cp1251"), bytes ("Строка2", "cp1251")]
f.writelines (arr)
f.close()
```

– **writable ()** - возвращает **True**, если файл поддерживает запись, и **False** - в противном случае:

```
f = open (r"file.txt", "r") #  Открываем файл для чтения
print(f.writable ())
False
f = open (r"file.txt", "w") # Открываем файл для записи
```

```
print(f.writable ())
True
```

– **read([<Количество>])** - считывает данные из файла. Если файл открыт в текстовом режиме, то возвращается строка, а если в бинарном - последовательность байтов. Если параметр не указан, возвращается содержимое файла от текущей позиции указателя до конца файла:

```
# Текстовый режим
with open(r"file.txt", "r", encoding="cp1251") as f:
        f.read()
'Строка1\nСтрока2'
# Бинарный режим
with open(r"file.txt", "rb") as f:
       f.read()
b'\xd1\xf2\xf0\xee\xea\xe01\n\xd1\xf2\xf0\xee\xea\xe02'
```

Если в качестве параметра указать число, то за каждый вызов будет возвращаться указанное количество символов или байтов. Когда достигается конец файла, метод возвращает пустую строку.

Пример:

```
# Текстовый режим
f = openopen(r"file.txt", "r", encoding="cp1251")
print(f.read(8)) # Считываем 8 символов
'Строка1\n'
print(f.read(8)) # Считываем 8 символов
'Строка2'
print(f.read(8)) # Достигнут конец файла
''
f.close ()
```

– **readline ([<Количество>])** - считывает из файла одну строку при каждом вызове. Если файл открыт в текстовом режиме, то возвращается строка, а если в бинарном - последовательность байтов. Возвращаемая строка включает символ перевода строки. Исключением является последняя строка - если она не завершается символом перевода строки, то таковой добавлен не будет. При достижении конца файла возвращается пустая строка.

Пример:

```
# Текстовый режим
f = open(r"file.txt", "r", encoding="cp1251")
print(f.readline (), f.readline ())
('Строка1\n', 'Строка2')
print(f.readline ()) # Достигнут конец файла
''
f.close ()
# Бинарный режим
f = open(r"file.txt", "rb")
print(f.readline (), f.readline ())
(b'\xd1\xf2\xf0\xee\xea\xe01\r\n', b'\xd1\xf2\xf0\xee\xea\xe02')
```

```
print(f.readline ()) # Достигнут конец файла
b''
f.close ()
```

Если в необязательном параметре указано число, то считывание будет выполняться до тех пор, пока не встретится символ новой строки **(\n)**, символ конца файла или из файла не будет прочитано указанное количество символов. Иными словами, если количество символов в строке меньше значения параметра, то будет считана одна строка, а не указанное количество символов, а если количество символов в строке больше, то возвращается указанное количество символов.

Пример:

```
f = open(r"file.txt", "r", encoding="cp1251")
print(f.readline (2) , f.readline (2))
('Ст', 'ро')
print(f.readline (100)) # Возвращается одна строка, а не 100 символов
'ка1\n'
f.close ()
```

– **readlines ()** - считывает все содержимое файла в список. Каждый элемент списка будет содержать одну строку, включая символ перевода строки. Исключением является последняя строка. Если она не завершается символом перевода строки, то символ перевода строки добавлен не будет. Если файл открыт в текстовом режиме, то возвращается список строк, а если в бинарном - список объектов типа **bytes**.

Пример:

```
# Текстовый режим
with open(r"file.txt", "r", encoding="cp1251") as f:
        f.readlines()
['Строка1\n', 'Строка2']
# Бинарный режим
with open(r"file.txt", "rb") as f:
        f.readlines()
[b'\xd1\xf2\xf0\xee\xea\xe01\r\n', b'\xd1\xf2\xf0\xee\xea\xe02']
```

– **__next__()** - считывает одну строку при каждом вызове. Если файл открыт в текстовом режиме, возвращается строка, а если в бинарном - последовательность байтов. При достижении конца файла возбуждается исключение **StopIteration**.

Пример:

```
# Текстовый режим
f = open(r"file.txt", "r", encoding="cp1251")
f.__next__(),   f.__next__()
('Строка1\n', 'Строка2')
f.__next__() # Достигнут конец файла
Traceback (most recent call last):
  File "<pyshell#59>", line 1, in <module>
    f.__next__() # Достигнут конец файла
```

```
StopIteration
f.close ()
```

Благодаря методу **__next__()** можно перебирать файл построчно с помощью цикла **for**. Цикл **for** на каждой итерации будет автоматически вызывать метод **__next__()**. Для примера выведем все строки, предварительно удалив символ перевода строки:

```
f = open(r"file.txt", "r", encoding="cp1251")
for line in f: print (line.rstrip ("\n"),   end=" ")
Строка1 Строка2
f.close ()
```

– **flush ()** - принудительно записывает данные из буфера на диск.

Рассмотрим оставшиеся методы, применяемые для работы с файлами.

– **fileno()** - возвращает целочисленный дескриптор файла. Возвращаемое значение всегда будет больше числа 2, т. к. число 0 закреплено за стандартным вводом **stdin**, 1 - за стандартным выводом **stdout**, а 2 - за стандартным выводом сообщений об ошибках **stderr**.

Пример:

```
f = open(r"file.txt", "r", encoding="cp1251")
print(f.fileno()) # Дескриптор файла
3
f.close ()
```

– **truncate ([<Количество>])** - обрезает файл до указанного количества символов (если задан текстовый режим) или байтов (в случае бинарного режима). Метод возвращает новый размер файла.

Пример:

```
f = open(r"file.txt", "r+", encoding="cp1251")
print(f.read())
'Строка1\nСтрока2'
print(f.truncate (5))
5
f.close ()
with open(r"file.txt", "r", encoding="cp1251") as f:
        f.read()
'Строк'
```

– **tell ()** - возвращает позицию указателя относительно начала файла в виде целого числа. Обратите внимание на то, что в **Windows** метод **tell()** считает символ **\r** как дополнительный байт, хотя этот символ удаляется при открытии файла в текстовом режиме.

Пример:

```
with open(r"file.txt", "w", encoding="cp1251") as f:
        f.write("String1\nString2")
15
f = open(r"file.txt", "r", encoding="cp1251")
f.tell() # Указатель расположен в начале файла
0
```

```
f.readline()  # Перемещаем указатель
'String1\n'
f.tell () #  Возвращает 9 (8 + '\r'), а не 8 !!!
9
f.close ()
```

Чтобы избежать этого несоответствия, следует открывать файл в бинарном режиме, а не в текстовом:

```
f = open(r"file.txt", "rb")
print(f.readline())  # Перемещаем указатель
b'String1\r\n'
print(f.tell ()) # Теперь значение соответствует
9
f.close ()
```

- **seek(<Смещение>[, <Позиция>])** - устанавливает указатель в позицию, имеющую смещение **<Смещение>** относительно позиции **<Позиция>**. В параметре **<Позиция>** могут быть указаны следующие атрибуты из модуля **io** или соответствующие им значения:
- **io.SEEK_SET** или 0 - начало файла (значение по умолчанию);
- **io.SEEK_CUR** или 1 - текущая позиция указателя. Положительное значение смещения вызывает перемещение к концу файла, отрицательное - к его началу;
- **io.SEEK_END** или 2 - конец файла.

Выведем значения этих атрибутов:

```
import io
print(io.SEEK_SET, io.SEEK_CUR, io.SEEK_END)
(0, 1, 2)
```

Пример использования метода seek():

```
import io
f = open(r"file.txt", "rb")
print(f.seek(9, io.SEEK_CUR))   #  9 байтов от указателя
9
print(f.tell())
9
print(f.seek(0, io.SEEK_SET)) # Перемещаем указатель в начало
0
print(f.tell())
0
print(f.seek (-9, io.SEEK_END)) # -9 байтов от конца файла
7
print(f.tell())
7
f.close ()
```

- **seekable ()** - возвращает **True**, если указатель файла можно сдвинуть в другую позицию, и **False** - в противном случае:

```
f = open (r"C:\temp\new\file.txt", "r")
```

```
print(f.seekable ())
True
```

Помимо методов, объекты файлов поддерживают несколько атрибутов:

- **name** - имя файла;
- **mode** - режим, в котором был открыт файл;
- **closed** - возвращает **True**, если файл был закрыт, и **False** - в противном случае.

Пример:

```
f = open(r"file.txt", "r+b")
print(f.name, f.mode, f.closed)
('file.txt', 'rb+', False)
f.close ()
print(f.closed)
True
```

- **encoding** - название кодировки, которая будет использоваться для преобразования строк перед записью в файл или при чтении. Атрибут доступен только в текстовом режиме. Обратите также внимание на то, что изменить значение атрибута нельзя, поскольку он доступен только для чтения.

Пример:

```
f = open (r"file.txt", "a", encoding="cp1251")
print(f.encoding)
'cp1251'
f.close ()
```

Стандартный вывод **stdout** также является файловым объектом. Атрибут **encoding** этого объекта всегда содержит кодировку устройства вывода, поэтому строка преобразуется в последовательность байтов в правильной кодировке. Например, при запуске с помощью двойного щелчка на значке файла атрибут **encoding** будет иметь значение **"cp866"**, а при запуске в окне **Python Shell** редактора **IDLE** – значение **"cp1251"**.

Пример:

```
import sys
print( sys.stdout.encoding)
'cp1251'
```

- **buffer** - позволяет получить доступ к буферу. Атрибут доступен только в текстовом режиме. С помощью этого объекта можно записать последовательность байтов в текстовый поток.

Пример:

```
f = open (r"file.txt", "ц", encoding="cp1251")
print(f.buffer.write (bytes ("Строка", "cp1251")))
6
f.close()
```

4.3 Функции для работы с каталогами

Для работы с каталогами используются следующие функции из модуля **os**:

– **getcwd ()** - возвращает текущий рабочий каталог. От значения, возвращаемого этой функцией, зависит преобразование относительного пути в абсолютный. Кроме того, важно помнить, что текущим рабочим каталогом будет каталог, из которого запускается файл, а не каталог с исполняемым файлом. Пример:

```
import os
os.getcwd () # Текущий рабочий каталог
'C:\\Python34'
```

– **chdir (<*Имя каталога*>)** - делает указанный каталог текущим:

```
os.chdir("C:\\book\\folder1\\")
os.getcwd () # Текущий рабочий каталог
'C:\\book\\folder1'
```

– **mkdir (<*Имя каталога*>[, <*Права доступа*>])** - создает новый каталог с правами доступа, указанными во втором параметре. Права доступа задаются восьмеричным числом (значение по умолчанию 0o777). Пример создания нового каталога в текущем рабочем каталоге:

```
os.mkdir ("newfolder") # Создание каталога
```

– **rmdir (<*Имя каталога*>)** - удаляет пустой каталог. Если в каталоге есть файлы или указанный каталог не существует, возбуждается исключение - подкласс класса **OSError**. Удалим каталог newfolder:

```
os.rmdir ("newfolder") # Удаление каталога
```

– **listdir (<Путь>)** - возвращает список объектов в указанном каталоге:

```
os.listdir ("C:\\book\\folder1\\")
['file1.txt', 'file2.txt', 'file3.txt', 'folder1', 'folder2']
```

– **walk ()** - позволяет обойти дерево каталогов. Формат функции:

walk(<*Начальный каталог*>[, topdown=True][, onerror=None] [, followlinks=False])

Как было отмечено на предыдущем шаге, функция **listdir()** возвращает список объектов в указанном каталоге. Проверить, на какой тип объекта ссылается элемент этого списка, можно с помощью следующих функций из модуля **os.path**:

– **isdir (<*Объект*>)** - возвращает **True**, если объект является каталогом, и **False** - в противном случае:

```
import  os.path
os.path.isdir(r"C:\book\file.txt")
False
os.path.isdir ("C:\\book\\")
True
```

– **isfile (<*Объект*>)** - возвращает **True**, если объект является файлом, и **False** - в противном случае:

```
os.path.isfile(r"C:\book\file.txt")
True
os.path.isfile("C:\\book\\")
False
```

– **islink (<*Объект*>)** - возвращает True, если объект является символической ссылкой, и **False** - в противном случае. Если символические ссылки не поддерживаются, функция возвращает **False**.

4.4 Сохранение объектов в файл. Модуль pickle

Сохранить объекты в файл и в дальнейшем восстановить объекты из файла позволяют модули **pickle** и **shelve**.

Модуль **pickle** предоставляет следующие функции:

– **dump (<*Объект*>, <*Файл*>[, <*Протокол*>][, fix_imports=True])** - производит сериализацию объекта и записывает данные в указанный файл. В параметре **<*Файл*>** указывается файловый объект, открытый на запись в бинарном режиме. Пример сохранения объекта в файл:

```
import pickle
f = open(r"file.txt", "wb")
obj = ["Строка", (2, 3)]
pickle.dump(obj , f)
f.close()
```

– **load ()** - читает данные из файла и преобразует их в объект. В параметре **<*Файл*>** указывается файловый объект, открытый на чтение в бинарном режиме. Формат функции:

```
load (<Файл> [, fix_imports=True] [, encoding="ASCII"]
   [, errors="strict"])
```

Пример восстановления объекта из файла:

```
f = open (r"file.txt", "rb")
obj = pickle.load (f)
print(obj)
['Строка', (2, 3)]
f.close ()
```

В один файл можно сохранить сразу несколько объектов, последовательно вызывая функцию **dump()**. Пример сохранения нескольких объектов приведен ниже.

```
obj1 = ["Строка", (2, 3)]
obj2 = (1, 2)
f = open(r"file.txt", "wb")
pickle.dump (obj1, f)   # Сохраняем первый объект
pickle.dump (obj2, f)   # Сохраняем второй объект
f.close ()
```

Для восстановления объектов необходимо несколько раз вызвать функцию **load()**:

```
f = open (r"file.txt", "rb")
obj1 = pickle.load(f)   # Восстанавливаем первый объект
obj2 = pickle.load(f)   # Восстанавливаем второй объект
print(obj1, obj2)
(['Строка', (2, 3)], (1, 2))
```

```
f.close ()
```

Сохранить объект в файл можно также с помощью метода **dump(<*Объект*>)** класса **Pickler**. Конструктор класса имеет следующий формат:

```
Pickler(<Файл>[, <Протокол>][, fix_imports=True])
```

Пример сохранения объекта в файл:

```
f =  open(r"file.txt", "wb")
obj = ["Строка", (2, 3)]
pkl = pickle.Pickler (f)
pkl.dump(obj)
f.close ()
```

Восстановить объект из файла позволяет метод **load()** из класса **Unpickler**. Формат конструктора класса:

```
  Unpickler(<Файл>[, fix_imports=True][, encoding="ASCII"]
    [, errors="strict"])
```

Пример восстановления объекта из файла:

```
f = open (r"file.txt", "rb")
obj = pickle.Unpickler(f).load()
print(obj)
['Строка', (2, 3)]
f.close()
```

Модуль **pickle** позволяет также преобразовать объект в последовательность байтов и восстановить объект из таковой. Для этого предназначены две функции:

- **dumps (<*Объект*>[, <*Протокол*>] [, fix_imports=True])** - производит сериализацию объекта и возвращает последовательность байтов специального формата. Формат зависит от указанного протокола - числа от 0 до 4 в порядке от более старых к более новым и совершенным. По умолчанию используется протокол 4, поддержка которого появилась в Python 3.4. Пример преобразования списка и кортежа:

```
obj1 = [1, 2, 3, 4, 5]   # Список
obj2 = (6, 7, 8, 9, 10)  # Кортеж
print(pickle.dumps (obj1))
b'\x80\x03]q\x00(K\x01K\x02K\x03K\x04K\x05e.'
print(pickle.dumps (obj2))
b'\x80\x03(K\x06K\x07K\x08K\tK\ntq\x00.'
```

- **loads (<*Последовательность байтов*>[, fix_imports=True][, encoding="ASCII"] [, errors="strict"])** - преобразует последовательность байтов специального формата в объект. Пример восстановления списка и кортежа:

```
print(pickle.loads(b'\x80\x03]q\x00(K\x01K\x02K\x03K\x04K\x05e.'))
[1, 2, 3, 4, 5]
print(pickle.loads(b'\x80\x03(K\x06K\x07K\x08K\tK\ntq\x00.'))
(6, 7, 8, 9, 10)
```

В заключение этого шага дадим краткую характеристику использованных в перечисленных функциях параметров.

– **fix_imports=True** - если этот параметр будет иметь значение по умолчанию (**True**) и версия использованного протокола меньше 3, **pickle** будет пытаться сопоставить новые имена **Python 3.x** с именами старых модулей, используемыми в **Python 2.x**, так что поток данных **pickle** мог читаться с помощью **Python 2.x**;

– **encoding** - содержит обозначение использованной кодировки символов;

– **errors** - определяет способ обработки ошибок при преобразовании. Значение по умолчанию (**"strict"**) возбуждает исключение **UnicodeEncodeError**, также могут быть использованы значения **"replace"** (неизвестный символ заменяется символом вопроса) или **"ignore"** (неизвестные символы игнорируются).

4.5 Сохранение объектов в файл. Модуль shelve

Модуль **shelve** позволяет сохранять объекты под определенным ключом (задается в виде строки) и предоставляет интерфейс доступа, сходный со словарями. Для сериализации объекта используются возможности модуля **pickle**, а чтобы записать получившуюся строку по ключу в файл, применяется модуль **dbm**. Все эти действия модуль **shelve** производит самостоятельно.

Открыть файл с набором объектов поможет функция **open()**. Функция имеет следующий формат:

open(<*Путь к файлу*>[, flag="c"][, protocol=None][, writeback=False])

В необязательном параметре **flag** можно указать один из режимов открытия файла:

– **r** - только чтение;

– **w** - чтение и запись;

– **c** - чтение и запись (значение по умолчанию). Если файл не существует, он будет создан;

– **n** - чтение и запись. Если файл не существует, он будет создан. Если файл существует, он будет перезаписан.

Функция **open ()** возвращает объект, с помощью которого производится дальнейшая работа с базой данных. Этот объект имеет следующие методы:

– **close ()** - закрывает файл с базой данных. Для примера создадим файл и сохраним в нем список и кортеж:

```
import shelve              # Подключаем модуль
db = shelve.open("db1")    # Открываем файл
db["obj1"] = [1,2,3,4,5]   # Сохраняем список
db["obj2"] = (6,7,8,9,10)  # Сохраняем кортеж
print(db["obj1"], db["obj2"])     # Вывод значений
([1, 2, 3, 4, 5], (6, 7, 8, 9, 10))
db. close ()               # Закрываем файл
```

– **keys ()** - возвращает объект с ключами;

– **values ()** - возвращает объект со значениями;

– **items ()** - возвращает объект-итератор, который на каждой итерации генерирует кортеж, содержащий ключ и значение. Пример:

```
db = shelve.open ("db1")
print(db.keys(), db.values())
(KeysView(<shelve.DbfilenameShelf object at 0x0245D590>),
ValuesView(<shelve.DbfilenameShelf object at 0x0245D590>))
print(list (db.keys()) , list (db.values()))
(['obj1', 'obj2'], [[1, 2, 3, 4, 5], (6, 7, 8, 9, 10)])
print(db.items())
ItemsView(<shelve.DbfilenameShelf object at 0x0245D590>)
print(list (db.items()))
[('obj1', [1, 2, 3, 4, 5]), ('obj2', (6, 7, 8, 9, 10))]
db.close()
```

– **get (<*Ключ*>[, <*Значение по умолчанию*>])** - если ключ присутствует, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, то возвращается значение **None** или значение, указанное во втором параметре;

– **setdefault (<*Ключ*>[, <*Значение по умолчанию*>])** - если ключ присутствует, то метод возвращает значение, соответствующее этому ключу. Если ключ отсутствует, создается новый элемент со значением, указанным во втором параметре, и в качестве результата возвращается это значение. Если второй параметр не указан, значением нового элемента будет **None**;

– **pop (<*Ключ*>[, <*Значение по умолчанию*>])** - удаляет элемент с указанным ключом и возвращает его значение. Если ключ отсутствует, возвращается значение из второго параметра. Если ключ отсутствует, и второй параметр не указан, то возбуждается исключение **KeyError**;

– **popitem ()** - удаляет произвольный элемент и возвращает кортеж из ключа и значения. Если файл пустой, возбуждается исключение **KeyError**;

– **clear ()** - удаляет все элементы. Метод ничего не возвращает в качестве значения;

– **update ()** - добавляет элементы. Метод изменяет текущий объект и ничего не возвращает. Если элемент с указанным ключом уже присутствует, то его значение будет перезаписано. Форматы метода:

update (<*Ключ1*>=<*Значение1*>[, ..., <*КлючN*>=<*ЗначениеN*>])
update (<*Словарь*>)
update (<*Список кортежей с двумя элементами*>)
update (<*Список списков с двумя элементами*>)

Помимо этих методов можно воспользоваться функцией **len ()** для получения количества элементов и оператором **del** для удаления определенного элемента, а также операторами **in** и **not in** для проверки существования или несуществования ключа. Пример:

```
db = shelve.open("db1")
print(len(db))  # Количество элементов
2
print("obj1" in db)
```

```
True
del db["obj1"]   # Удаление элемента
print("obj1" in db)
False
print("obj1" not in db)
True
db.close()
```

В заключение дадим краткое описание использованных параметров. Параметр **protocol** определяет протокол функции **pickle()**. Более интересным является назначение параметра **writeback**. Если его значение равно **False** (по умолчанию), то измения элемента не будут сохранены автоматически, в отличие от значения **True**, однако в этом случае могут быть задействованы достаточно большие ресурсы, и программа будет работать медленно.

Рассмотрим эту особенность на конкретном примере:

```
db = shelve.open("db1") # Пареметр writeback=False
db["db1"] = [0, 1, 2]   # Добавим еще один элемент
print(db["db1"])            # Выведем его значение
[0, 1, 2]
db["db1"].append(3) # Добавили элемент в список
print(db["db1"])         # Выведем его значение. Добавления не произошло!!!
[0, 1, 2]
temp = db["db1"] # Поместим элемент в переменную
temp.append(3)   # Выполним здесь добавление...
db["db1"] = temp # ... и присваивание
print(db["db1"])       # Выведем его значение. Все получилось!!!
[0, 1, 2, 3]
db.close()
db = shelve.open("db1", writeback = True) # Пареметр writeback=True
print(db["db1"])         # Выведем значение элемента
[0, 1, 2, 3]
db["db1"].append(4) # Добавили элемент в список
print(db["db1"])         # Выведем его значение. Все получилось!!!
[0, 1, 2, 3, 4]
db.close()
```

4.6 Пример решения задачи на языке программирования Python

Задача. Разработайте программу, которая формирует список из пяти случайных элементов. Данные списка консервируются и записываются в двоичный файл. Во второй программе реализуйте чтение списка из файла, создайте функцию, которая осуществляет нахождение суммы сгенерированных элементов списка, находящихся в промежутке от 0 до 10 включительно. Результат выведите в текстовый файл.

Ниже приведен код программы, который осуществляет формирование списка и его консервацию.

Решение задачи:

```
import pickle, random
spisok=(5, 10, 9, 7, 40, 50, 70, 80, 1, 2)
chislo=random.sample(spisok,5)
zap=open("mass.dat","wb")
pickle.dump(chislo,zap)
zap.close()
```

Ниже приведен код программы, который осуществляет извлечение списка из файла, нахождение суммы его элементов и запись результата в текстовый файл.

```
import pickle
zap=open("mass.dat","rb+")
chislo=pickle.load(zap)
print(chislo)
zap.close()
def F_sum(*arg): #Функция подсчитывает сумму элементов
    sum=0
    for i in arg:
        if (i>=0) and (i<=10):
            sum+=i
    return sum
summa=F_sum(*chislo) #Вызов функции
print("\nСумма элементов >=0 и <=10 = ",summa)
summa_=str(summa) #Преобразование полученного результата в строковое значение
f=open("rezult.txt", "w+", encoding='utf-8') #Открываем файл на запись
f.write(summa_)
f.close()
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте текстовый файл, запишите в него построчно данные, которые вводит пользователь. Окончанием ввода служит пустая строка.

2 В текстовом файле посчитайте количество строк, а также для каждой отдельной строки определите количество в ней символов и слов.

3 В текстовый файл построчно записаны фамилия и имя учащихся класса и его оценка за контрольную. Выведите на экран всех учащихся, чья оценка меньше 3 баллов и посчитайте средний балл по классу.

4 Подсчитайте количество сдвоенных символов 'aa', 'oo', 'кк' в тексте, расположенном в текстовом файле, затем удалите повторяющийся символ. Полученную строку запишите в другой файл.

5 Подсчитайте число слов в предложении, записанном в текстовом файле.

6 Найдите в текстовом файле самое длинное и самое короткое слово.

7 В файле записаны целые числа. Найдите максимальное и минимальное число и запишите результат в новый файл.

8 В файле записаны целые числа, каждое число начинается с новой строки. Отсортируйте их по возрастанию суммы цифр и запишите в новый файл.

9 Из строки, расположенной в текстовом файле, исключите все символы, входящие в нее более одного раза. Полученную строку сохраните в другом файле.

10 В последовательности символов, заданной в текстовом файле, подсчитайте общее количество символов '+','-', '*'.

11 В текстовом файле, в предложении, содержащем не менее двух слов, поменяйте местами первое и последнее слова, а затем результат сохраните в другом файле.

12 В текстовом файле две строки текста. Необходимо сформировать третью строку, состоящую из символов, входящих одновременно в обе исходные строки, и дописать ее в текстовый файл.

13 Откорректируйте текст, расположенный в текстовом файле, заменив в нем все вхождения одной буквы на другую. Результат запишите в другой файл.

14 Перепишите текстовый файл таким образом, чтобы все слова исходного текста были перевернуты. Результат запишите в другой файл.

15 Из текста, расположенного в файле, удалите лишние пробелы, разделяющие слова.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте бинарный файл и запишите в него любую строку, предварительно преобразовав её в последовательность байтов.

2 Прочитайте содержимое созданного бинарного файла и выведите его на экран.

3 Создайте бинарный файл с последовательностью байтов от 1 до 50 и убедитесь, что файл содержит ровно 50 байтов.

4 Определите, сколько раз в бинарном файле встречается конкретное значение байта (например, 10).

5 Считайте первые 10 байтов из бинарного файла и выведите их на экран.

6 Создайте бинарный файл, содержащий числа от 100 до 120, и измените значение байта с индексом 5 на 255.

7 Скопируйте содержимое одного бинарного файла в другой.

8 Создайте бинарный файл, содержащий 20 случайных байтов. Найдите минимальное и максимальное значение байта.

9 Запишите в бинарный файл текстовую строку с помощью кодировки UTF-8, затем прочитайте её обратно.

10 Создайте бинарный файл с массивом чисел и посчитайте их сумму, прочитав данные как байты.

11 Создайте бинарный файл и запишите туда все чётные числа от 2 до 100.

12 Создайте бинарный файл с последовательностью байтов от 1 до 10, затем поменяйте местами первый и последний байт.

13 Прочитайте бинарный файл и создайте новый файл, в котором все байты увеличены на 1.

14 Создайте бинарный файл, содержащий последовательность чисел от 1 до 100, и определите количество байтов, значения которых кратны 5.

15 Создайте бинарный файл со случайной последовательностью байтов и сформируйте новый файл, содержащий только уникальные байты из исходного файла.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Охарактеризуйте функции из модуля os для доступа к файлам в языке программирования Python.

7.2 Перечислите значения, которые может принимать необязательный параметр mode в функции open() в языке программирования Python.

7.3 Приведите синтаксис открытия файла в языке программирования Python.

7.4 Опишите функции для работы с файлами в языке программирования Python.

7.5 Опишите функции для работы с каталогами в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.